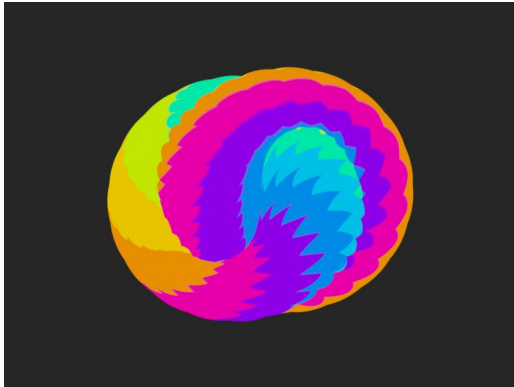


COMS W4156 Advanced Software Engineering (ASE)

October 19, 2023



Agenda

1. Continuous Integration
2. CI demo: Rishabh Srivastava
3. Discuss upcoming First Assessment



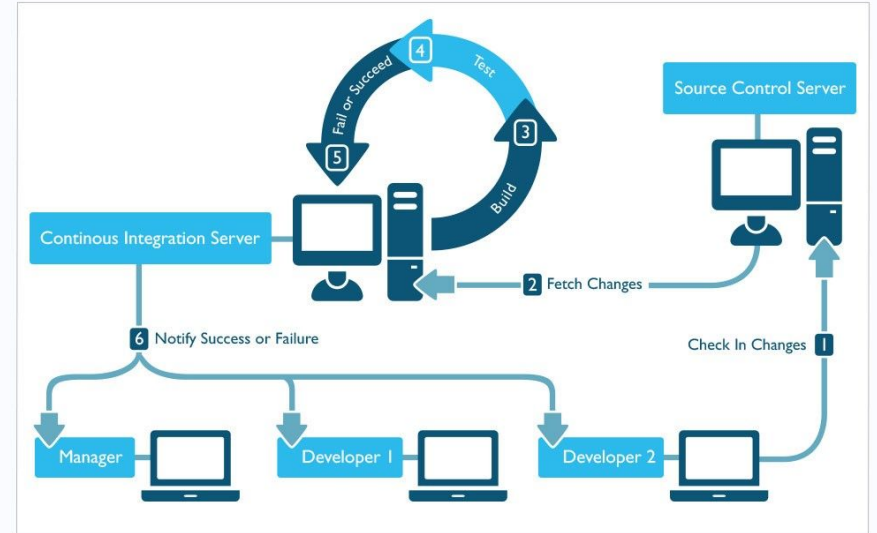
Continuous Integration ([CI](#))

Tools like [Jenkins](#), [Travis CI](#) and [Circle CI](#) hook into a version control system (like github) and automatically re-build (on designated build server) after every commit to the shared repository

Can also run CI on pre-commit (partial build on developer's machine) and/or on a timer (e.g., [nightly build](#) on build server)

Github provides built-in mechanisms supporting CI (and other capabilities) via [github actions](#)

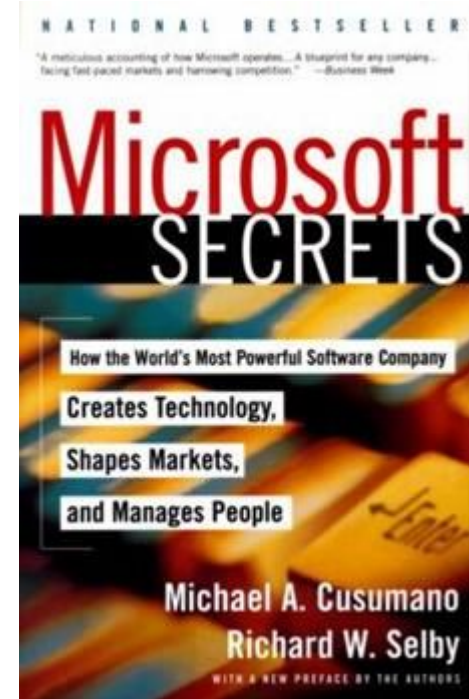
Goal is to detect errors asap after they are introduced



Don't Break The Build

Breaking the build = Errors detected during CI were traced to *your* commit

If all developers had fully tested their changes locally (on their own machine) prior to commit, and fixed any errors prior to commit, *in theory* CI should not find any new errors



In Theory vs. In Practice

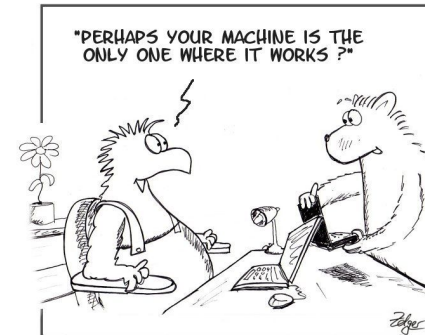
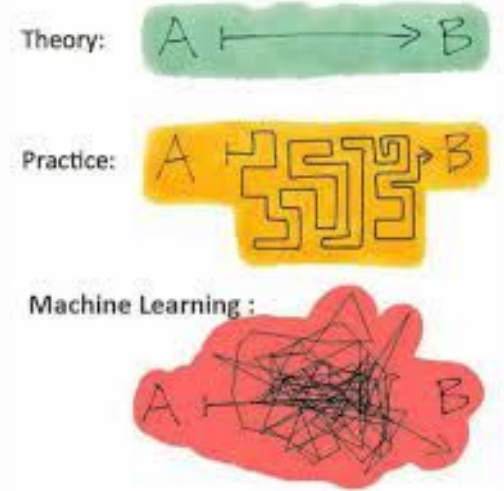
Developers test their own changes locally with only a subset of the full test suite

Sometimes just the unit tests for the changed units

Plus maybe integration tests involving the changed units

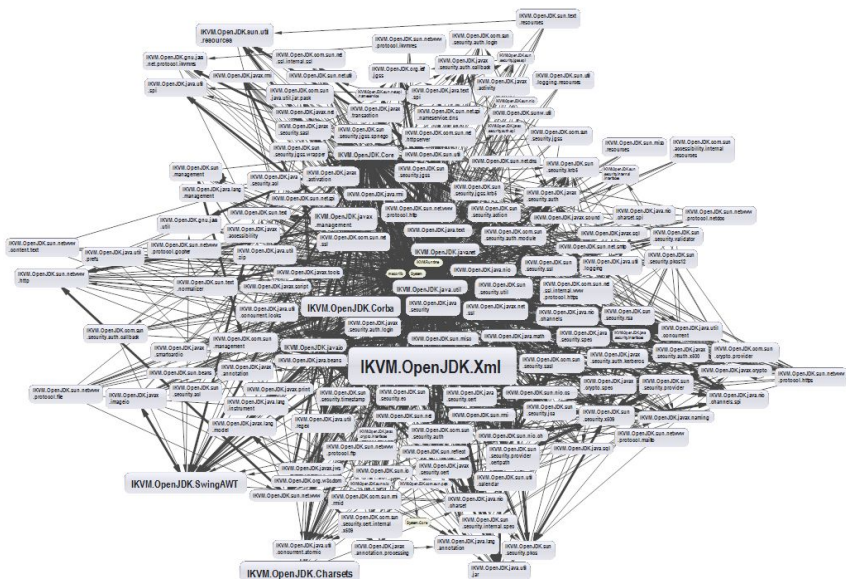
Developer machines typically do not have the resources to run the system's full regression test suite

And developers do not have the time to wait for full builds in a test environment



It works on my machine

I Thought Build Meant Compile



Build often refers to gathering all dependencies, followed by compile and link (compiled languages) or packaging together all the dependencies (non-compiled languages)

In CI context, “build” also runs testing and analysis tools

Which test cases actually run when may vary - in some organizations there are multiple levels of CI running designated tests

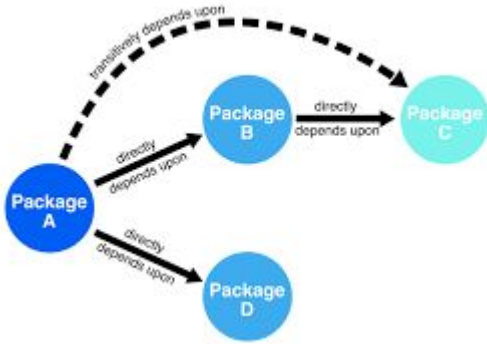
Test Selection

A common choice for commit-time testing is to employ a “test minimization” strategy that selects only new/modified test cases plus those existing test cases whose results (pass or fail) could have been affected by the committed code changes ([change analysis](#))



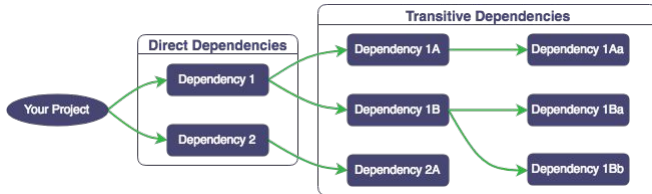
This may be a relatively small subset of all the tests, but there’s a cost to accurately computing the affected tests without actually running any tests

Change Analysis



Track all code c exercised by each test t during *previous* testing

If the commit changes c , or changes any other code that c depends on directly or indirectly (determined by static analysis), then re-run t



It may be less expensive to run full regression testing than to run full static analysis

Regression Testing

Regression testing is the typical choice for nightly builds

A *regression* is when fixing a bug or adding a new feature causes previously working code to break, or “regress”, sometimes code that seems completely unrelated in another part of the codebase (because there was a hidden dependency)

Regression:
"when you fix one bug, you
introduce several newer bugs."



Detecting Regressions



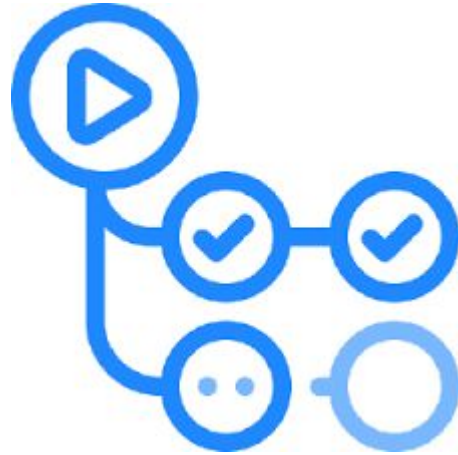
To detect all regressions, you need to re-run *all* test cases that previously passed - and also all tests that previously failed, in case they now fail in different ways or (unexpectedly) pass. In other words, the whole test suite!

Of course, if the code change fixed a bug, you would expect the corresponding test case(s) that detected the bug to now pass. The developers should run those previously failed tests before committing

But other test cases that do not seem relevant may also now pass due to dependencies in the application (incomplete or incorrect change analysis) or in the test cases themselves (hidden dependencies)

Agenda

1. Continuous Integration
2. **CI demo: Rishabh Srivastava**
3. Discuss upcoming First Assessment



CI demo: Rishabh Srivastava

[demo slides](#)

[demo repo](#)

Agenda

1. Continuous Integration
2. CI demo: Rishabh Srivastava
3. **Discuss upcoming First Assessment**



Upcoming Assignments

- [First Iteration](#) due tonight by 11:59pm
- [First Iteration Demo](#) due next Thursday, October 26, by 11:59pm

First Assessment will be 12:01am Friday October 27 through 11:59pm Monday October 30
(A few students will have a different time window)



- [Second Iteration](#) due Thursday, November 30, by 11:59pm
- [Second Iteration Demo](#) due Thursday, December 7, by 11:59pm



First Assessment

You will have four days (minus two minutes) to complete the assessment, which should take 4-6 hours total. The assessment will not be timed, you can spend as much or as little time as you want within the four days

Covers some of the material discussed in class from beginning of semester through today (October 19)

Will not cover any new material introduced next week

Includes two ungraded questions along with the graded questions

One ungraded question asks for feedback on your team members - answering this is **mandatory**, we will not grade your assessment otherwise and you will get 0

The other ungraded question asks for feedback on this class - answering this is **optional**

Mandatory Ungraded Question



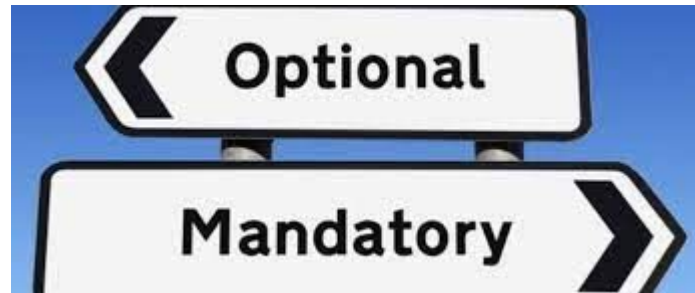
You have 100 total points to allocate among the members of your team. **Assign some number of points (possibly 0) to each member of your team based on what you believe was that student's relative participation in the team project thus far, and explain your scores.** Make sure the total is exactly 100. Remember to include yourself! For allocating points, consider level of effort and contributions for anything relevant to the team project. Note equal participation is not the same thing as equal lines of code produced. Please do not predict the future, consider only the contributions to the team project as of the date of writing. It is very important that you explain the scores you give to each student, including yourself: *If you do not provide meaningful explanations, then your response will be considered incomplete and you will receive 0 overall on the assessment.*

For example, in a team where all team members participated more or less equally, the scores would be: Cora 25, Kaedi 25, Kaylah 25, Nina 25, and your explanation would describe who did which part(s) of the team project effort thus far.

In a team with a substantial disparity in contributions, however, the scores might be: Hawkeye 35, Captain Marvel 35, Hulk 25, Thor 5, and your explanation might describe Hawkeye and Captain Marvel as doing most of the work, and Hulk helped, but Thor rarely showed up for team meetings and did not seem interested in the project. Again, your explanation should describe who did what, or who did not do what, as the case may be.

Optional Ungraded Question

Is there anything that you think should have been covered already but wasn't? Is there anything that was covered unnecessarily or should have been put off until later? Is there anything that should have been presented in a different way? Is there anything else you would change about the course? Please explain. Suggestions for the remainder of this course will be greatly appreciated. In addition to submitting comments and suggestions here as part of your first assessment, you can also post anonymously on Edstem. (And please also submit the SEAS evaluation.)



Upcoming Assignments

- [First Iteration](#) due tonight by 11:59pm
- [First Iteration Demo](#) due next Thursday, October 26, by 11:59pm



First Iteration vs. First Iteration Demo

By the time you submit your First Iteration assignment, all the pieces should be in place in your github repo (possibly preliminary versions of some of the pieces)

By the time you submit your First Iteration Demo assignment, your service should work, you should be able to show that it works, and your Mentor should be able to "drive"



First Iteration all the pieces in place

Your README should document your API (or link to your external API docs). It should also contain instructions (that work!) for how to build, run and test your service

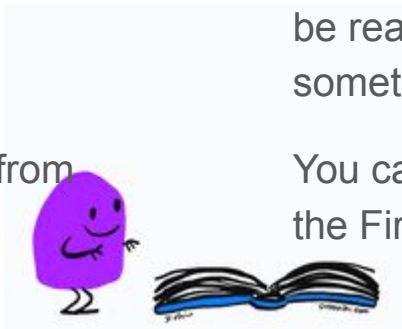
Your repo should contain system-level tests exercising your API entry points "over the network" - this can be localhost but should not be in-process or otherwise assume same machine

Some of the API tests should involve persistent data and pretend to come from different clients

Your repo should contain unit tests for most major methods, with setup/teardown and mocks when applicable, that run "push button" from a unit testing tool

Your repo should contain some style-checker reports and most of its complaints should have been fixed. Your code should be commented, use mnemonic identifiers and be readable. Commit messages should say something meaningful about what changed

You can continue updating all of these until the First Iteration Demo and beyond



Ask Me Anything